T1-02: Deep Learning Foundations

DLS 2026

Draft — 11 October 2026

Learning objective. Students will trace one prediction through representation, loss, gradient, and parameter update; explain why a train/test split and gradient check are necessary; and identify a failure that an optimization curve alone cannot diagnose.

1 From examples to a parameterized prediction

A supervised learner receives examples (x, y) and a family of functions f_θ. A representation transforms raw input into features useful for a task; a model maps those features to predictions; a loss measures disagreement with targets. Deep learning composes many simple differentiable transformations, enabling representations to be learned jointly with the predictor rather than fixed by hand [1, 2].

For binary classification, a linear logit $z = w^T x + b$ becomes a probability $p = \sigma(z) = 1/(1 + e^{-z})$. Cross-entropy loss for label $y \in \{0, 1\}$ is $\ell = -[y \log p + (1 - y) \log(1 - p)]$. The derivative with respect to the logit is $p - y$. This compact result says: if a positive example receives too little probability, increase its logit; if a negative example receives too much, decrease it.

2 Gradients make learning local

The chain rule propagates a loss signal backward through the computation graph. For the logistic example, $\nabla_w \ell = (p - y)x$ and $\partial \ell / \partial b = p - y$. Gradient descent updates $w \leftarrow w - \eta \nabla_w \ell$. Backpropagation is not a separate learning rule; it is an efficient application of the chain rule to shared intermediate computations.

A gradient is only a local slope. Learning also depends on initialization, scale, learning rate, batch sampling, regularization, data quality, and architecture. A decreasing training loss can coexist with poor test performance, label leakage, class imbalance, or a misaligned objective.

3 Worked example

Let $x = (2, -1)$, $w = (0.1, 0.2)$, $b = 0$, and $y = 1$. Then $z = 0$, $p = 0.5$, and $p - y = -0.5$. Thus $\nabla_w \ell = (-1, 0.5)$ and $\partial \ell / \partial b = -0.5$. With $\eta = 0.1$, the update is $w = (0.2, 0.15)$ and $b = 0.05$. The new logit for this example is 0.3, so its predicted probability increases. This is one local improvement, not proof of generalization.

4 Representations and nonlinearity

A single linear classifier can separate classes only with a linear boundary in its input features. Hidden layers alternate affine transformations with nonlinear activations such as ReLU, $\max(0, a)$. This permits the model to construct intermediate features that make later decisions simpler. For example, an image model may first form edge-like responses, combine them into textures or parts, and use those features

for a class decision. These are useful descriptions of a computation, not proof that every hidden unit has a stable human interpretation.

Composition also changes the optimization problem. Gradients multiply through layers, so poor scaling or saturated activations can make signals extremely small or large. Modern practice therefore pays attention to initialization, normalization, residual paths, optimizer choice, batch size, and learning-rate schedules. These techniques make optimization more reliable; they do not remove the need to test the task and data pipeline.

5 What a test can and cannot establish

A test set estimates performance on data sampled from its stated population. It does not automatically answer whether the labels capture the desired outcome, whether the deployment distribution will match, or whether rare errors are acceptable. For a medical triage screen, a small false-negative rate may matter much more than average accuracy. For an educational tool, an answer that is fluent but reinforces a misconception may be more damaging than a concise abstention.

Gradient checks provide a complementary lesson in scope. They can establish that a small implementation's local derivatives agree with numerical estimates. They cannot establish that the model will converge quickly, generalize, use an appropriate objective, or behave fairly. Good engineering uses narrow tests for narrow claims and does not overstate what a passing test means.

6 Validation and limits

Hold out examples that were not used to choose parameters or tune decisions. Keep preprocessing inside the training split to avoid leakage. A finite-difference gradient check—compare an analytic derivative to $[L(\theta+\epsilon)-L(\theta-\epsilon)]/(2\epsilon)$—tests implementation logic on a small problem, but is too expensive for routine training. Optimization can be numerically correct while the task definition is socially or scientifically wrong.

7 An experiment plan students can inspect

A defensible small experiment begins with a claim and a comparison. For instance: "On this fixed synthetic classification task, adding a hidden ReLU layer improves held-out cross-entropy relative to logistic regression under the same split and compute budget." The plan records the data generator, split seed, architectures, optimization settings, stopping rule, and metrics. It then reports both learning curves and a small table of mistakes. If the hidden model wins only on training loss, the claim is not supported. If it wins on the held-out set but fails badly on one subgroup deliberately included in the generator, the aggregate result needs qualification.

This practice is useful because models can improve for accidental reasons. A preprocessing transform computed from all examples leaks test information. Repeatedly choosing hyperparameters against one validation set eventually overfits that set. A benchmark can reward a shortcut correlated with labels. Separating training, validation, and final test decisions does not eliminate these risks, but it creates a record of what evidence each decision was allowed to see.

8 From derivative to debugging

When a training run fails, start with a tiny case. Confirm shapes and units, overfit a handful of examples, compare loss before and after one update, and run a finite-difference check on a few parameters. Then inspect data labels and splits before escalating model size. This order is not glamorous, but it separates

implementation mistakes from optimization choices and from task-design errors. A larger model may hide a bug for a while; it rarely explains it.

9 Takeaway

Representations, losses, gradients, and updates form a causal chain: chosen data and objective define what improvement means; gradients estimate how parameters change that objective; validation asks whether the resulting behavior transfers beyond the examples used to fit it.

References

[1] Rumelhart, Hinton, and Williams (1986). *Learning representations by back-propagating errors.*

[2] Goodfellow, Bengio, and Courville (2016). *Deep Learning.*